NOVAMART FIELD LAB / START

Build, trace and troubleshoot an AI support system

A step-by-step learning workbook

Prepared for **Muhammad Waqas** | NovaMart project | 24 October 2026

Learn how a customer request travels through five Strands agents, retrieves verified information, records findings, applies safety rules, and becomes a customer-facing answer on Amazon Bedrock AgentCore.

Learning deliverable	What is inside
Architecture	3D-style model, agent graph and request walkthrough
Practical labs	Foundation, agent tools, RAG, Guardrail, Memory, deployment and traces
Real evidence	Your deployment, model-access, rounding, 120/120 and X-Ray screenshots
Troubleshooting	Symptoms, diagnostic commands, causes and verification checkpoints

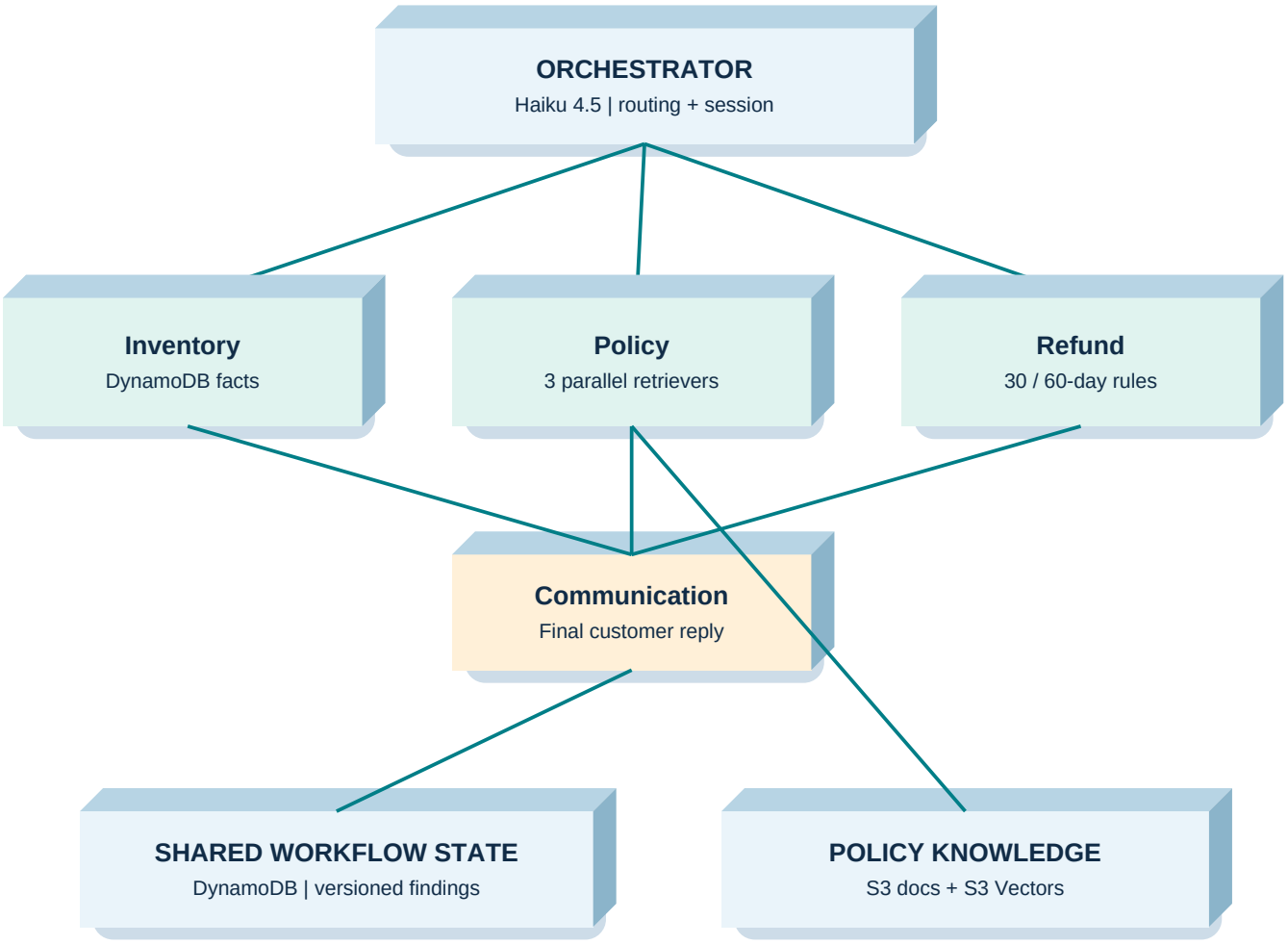
How to use this workbook

Read the purpose, follow the numbered lab steps, run only the commands appropriate to your environment, and record the checkpoint. The commands are a learning runbook; they have not been executed again for this document.

Scope: This documents your classroom implementation and observed test evidence. A 120/120 rubric score does not prove every production property. Real customer authentication, payment-provider integration and persistent memory event wiring require additional work.

See the whole system in 3D

CONCEPTUAL 3D-STYLE VIEW - not a screenshot or runtime service map



Read the model from top to bottom

The Orchestrator chooses a specialist. Inventory gathers customer/order facts. Policy retrieves policy text. Refund evaluates eligibility. Communication writes the final reply. All findings accumulate in one session record.

The illustration groups logical responsibilities. Routing is conditional: a math request goes directly to Communication; a policy question uses Policy; an order/return request uses Inventory then Refund. Workers run within the application, not as five separately deployed runtimes.

Tools: what to open and why

Tool or service	Purpose and when you use it
AWS Console	Visual management: CloudFormation, S3, Knowledge Bases, Bedrock playground and X-Ray map.
CloudShell + AWS CLI	Authenticated terminal for stack deployment, diagnostics and running Python.
Python 3.12 + venv + uv	Isolate dependencies; uv installs packages and supports CLI application packaging.
Strands SDK	Agent reasoning loop, BedrockModel configuration, Python tools and worker calls.
boto3 / botocore	Application calls to DynamoDB, Bedrock, Memory, logs and tracing APIs.
AgentCore CLI 0.30.0	Project helper configures the runtime and deploys through CDK/CloudFormation.
Node.js + npm + TypeScript	Build-time requirements for this CLI/CDK version; tsc compiles CDK code.
DynamoDB	Orders, customer profiles and shared optimistic-locking WorkflowState.
S3 + S3 Vectors	S3 stores original policy text; S3 Vectors holds embeddings in three indexes.
Bedrock Knowledge Bases	Chunk, embed, index and retrieve relevant policy passages.
Guardrails + Memory	Filter model inputs/outputs; provision session-summary memory.
CloudWatch + X-Ray	Logs explain failures; traces show request relationships and latency.

Two meanings of tool: Developer tools are commands you operate. Agent tools are Python functions exposed with @tool that the model selects. A tool docstring describes the function, parameters and result to the model.

Lab 1 - verify the environment

01 / Open CloudShell
Select us-east-1 in AWS Console. Open CloudShell and change into the project directory.

02 / Verify identity
Use STS to confirm the account. This is a read-only identity check; never paste access keys or session tokens.

03 / Activate Python
Use the project virtual environment. Verify that Python and required tools resolve from the expected locations.

04 / Inspect configuration
Run config.py. Foundation resource names should be populated; runtime fields can be blank before initial deployment.

```
cd "$HOME/novamart-starter"
source .venv/bin/activate
aws sts get-caller-identity --output json
python --version
command -v python
command -v uv
agentcore --version
python config.py
```

Fresh local environment only

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

If using a uv-created environment without pip: **uv pip install --python .venv/bin/python -r requirements.txt**. Do not add --user inside a virtual environment.

Checkpoint

Record your account, region, Python version, CLI version and project path. Your screenshots showed root identity during troubleshooting; prefer a suitably authorized development role for ongoing work.

Lab 2 - foundation infrastructure

01 / Review the template
Open infrastructure/starter_stack.yaml. Identify the three DynamoDB tables, policy bucket, vector bucket/indexes, runtime role and log group.

02 / Deploy once
Use CloudFormation Console upload or the CLI command below. Wait for CREATE_COMPLETE. For an existing stack, inspect its state before redeploying.

03 / Seed sample data
Run the supplied seed script after the stack is ready. Avoid reseeding a completed exercise without reviewing its effects on mock orders.

04 / Load environment defaults
Copy .env.example only on first setup; do not overwrite your populated .env. Run config.py again.

```
aws cloudformation deploy \
  --template-file infrastructure/starter_stack.yaml \
  --stack-name udacity-agentcore \
  --capabilities CAPABILITY_NAMED_IAM \
  --region us-east-1

python infrastructure/seed_data.py
python config.py
```

Resource	Role in a request
orders	Composite key: customer_id + order_id. Never look up an order using order_id alone.
customers	Customer tier and profile facts; PolicyAgent cannot determine a customer account tier.
workflow-state	session_id key; findings and version counter shared by workers.
policy documents	policies/returns/, policies/shipping/, policies/warranty/.
vector indexes	returns-policy-index, shipping-policy-index, warranty-policy-index.

Checkpoint

Verify orders/customers exist, each S3 prefix contains text documents, all three vector indexes exist, and the configuration resolves the foundation role.

Lab 3 - build the worker tools

Agent / temperature	Python tools and responsibility
Inventory / 0.1	check_order_status(customer_id, order_id); get_customer_tier; list_customer_orders. Gather facts only.
Refund / 0.1	get_inventory_context; initiate_refund. Recheck authoritative order/customer facts before a mutation.
Policy / 0.2	search_all_policies. Coordinate retrieval and synthesize a grounded policy answer.
Communication / 0.3	get_full_workflow_context. Read findings and draft the final empathetic reply.
Orchestrator / 0.0	initialize_session and four route_to_* tools. Delegate; Communication is the final tool call.

01 / Open the implementation
Edit only src/agent_orchestrator.py for the assignment TODOs. Keep the supplied support modules and official tests unchanged.

02 / Build each Agent
Use config.WORKER_MODEL_ID for workers and retrievers; use config.ORCHESTRATOR_MODEL_ID for the router. Do not rely on SDK default models.

03 / Document every tool
Describe purpose, Args and Returns. Return useful structured facts or a clear missing-data result.

04 / Verify routing
Run task2 tests and inspect the trace, not only the final answer.

```
python tests/test_agent.py task2
python src/demo.py
python src/agent_orchestrator.py chat
```

Production boundary: the mock initiate_refund updates an order to return_requested with a reference. That does not transfer money. A real refund requires a payment-provider transaction, authorization and audit trail.

Lab 4 - shared state and routing

Optimistic locking: a version is a concurrency check

Suppose Inventory and Policy both read version 2. Inventory writes with expected_version=2, so the record becomes version 3. Policy cannot blindly write its stale version 2 result. It must reread and safely retry or report a conflict. This prevents lost updates.

Routing rule	Required behavior
Every turn	Initialize the session first. Bind the request to its customer/session.
Order status / return / refund	Inventory first, then Refund. Status-only questions must not initiate a return.
Policy meaning	PolicyAgent retrieves policy text.
Account tier	InventoryAgent retrieves the actual customer profile.
Arithmetic	CommunicationAgent; skip Inventory, Policy and Refund.
Final step	CommunicationAgent composes the response. Orchestrator relays it.

Hands-on state lab

01 / Use a new session
Run a scripted scenario and record the session ID printed in the terminal.

02 / Open DynamoDB
Open workflow-state and locate the exact session_id.

03 / Inspect worker columns
Match inventory_agent, refund_agent, policy_agent and communication_agent to the route used.

04 / Inspect version
Confirm successful writes advance the version. A math-only request should not contain invented inventory findings.

Eligibility lab: Standard=30 days, Premium=60 days. Check the delivery timestamp and policy exclusions. If delivery evidence is missing, request review instead of inventing a date. The supplied seed data did not provide delivery dates, so review_required can be the honest outcome.

Lab 5 - create and sync three KBs

01 / Verify S3 sources
In S3, inspect the returns, shipping and warranty prefixes under policies/.

02 / Create self-managed KBs
In Bedrock Knowledge Bases, create one KB per domain. Select the existing S3 Vectors bucket from config.py, not a separate managed vector store.

03 / Select embeddings
Use Amazon Titan Embed Text v2: amazon.titan-embed-text-v2:0. Embedding dimensions must match the existing vector indexes.

04 / Select the matching index
Returns goes to returns-policy-index; repeat for shipping and warranty. Scope each source to its own prefix.

05 / Sync each source
Wait for ingestion COMPLETE. Empty retrieval often means wrong source prefix, missing sync, an ID mismatch or an ingestion failure.

06 / Save IDs and test
Populate RETURNS_KB_ID, SHIPPING_KB_ID and WARRANTY_KB_ID in .env; run task5.

```
python config.py
python tests/test_agent.py task5
```

Domain	Observed KB ID in this project
Returns	XCYTUZUYVQ
Shipping	BGRI6QGOYI
Warranty	7MUOQYLS0F

These are resource identifiers for your completed project, not reusable IDs for a new AWS account. A recreated lab gets new IDs. Original policy text stays in S3; the KB retrieves indexed passages through its vector store.

Lab 6 - parallel multi-agent RAG

What happens inside search_all_policies

PolicyAgent calls one tool. That tool submits a query to three complete Strands retriever agents concurrently. Each retriever has one domain-specific KB retrieval tool. The coordinator receives all three results and synthesizes them.

```
with ThreadPoolExecutor(max_workers=3) as pool:
    futures = {
        pool.submit(returns_agent, query): "Returns",
        pool.submit(shipping_agent, query): "Shipping",
        pool.submit(warranty_agent, query): "Warranty",
    }
    for future in as_completed(futures):
        domain = futures[future]
        results[domain] = str(future.result())
```

This is a conceptual excerpt. The project implementation also handles trace/context propagation and retrieval failures. Retriever temperature=0.0; coordinator temperature=0.2.

Practical experiment

01 / Ask a cross-domain question
Example: What benefits do premium customers get for returns, shipping and warranty?

02 / Watch all three branches
Confirm each domain appears and returns non-empty passages. Completion order can vary.

03 / Compare elapsed time
Parallel wall time is roughly the slowest branch plus coordination overhead, rather than the sum of three branches.

04 / Check grounding
Read cited S3 passages. Disclose conflicting passages; never silently select a convenient policy.

Observed data caveat: starter documents disagreed on the Standard electronics warranty duration. A grounded answer should explain that conflict or limit the answer to consistent facts.

Lab 7 - guardrails and model access

Policy	Required configuration
Content	SEXUAL, VIOLENCE, HATE: HIGH; INSULTS, MISCONDUCT: MEDIUM.
PII	BLOCK cards and SSNs; ANONYMIZE email addresses and phone numbers.
Denied topics	Competitor products, price negotiations and legal threats.
Topic tier	STANDARD with cross-region profile us.guardrail.v1:0.
Words	Managed profanity list enabled; friendly input/output block messages.
Version	Publish with create_guardrail_version; use numbered version, not DRAFT.

Safety test matrix

Input type	Expected result
5 items at $29.99 with 10% off	Allowed arithmetic; do not classify as haggling.
Can you lower the advertised price for me?	Negotiation topic blocked.
Recommend a competitor product	Competitor topic blocked.
Legal-threat example	Legal-threat topic blocked.
Email/phone in text	Anonymized when detected, according to configured policy.

Test INPUT and OUTPUT independently with ApplyGuardrail. A guardrail is a model-content control; customer ownership checks and DynamoDB write conditions still belong in application code.

Model access diagnostic

```
aws bedrock get-foundation-model-availability \
  --model-id anthropic.claude-haiku-4-5-20251001-v1:0 \
  --region us-east-1 --no-cli-pager
```

Repeat for Sonnet 4.5 using anthropic.claude-sonnet-4-5-20250929-v1:0. Inspect agreement, entitlement, authorization and region status. Your account eventually showed AVAILABLE and AUTHORIZED for both.

Lab 8 - Memory and deployment

Memory is a separate capability

configure_memory provisions a summaryMemoryStrategy resource with seven-day event retention and waits for ACTIVE. WorkflowState stores task findings. These are different stores. Creating a Memory resource alone does not prove conversation recall: persistent recall also needs application event writes and retrieval integrated with actor/session identifiers.

01 / Complete prerequisites
Foundation, synced KB IDs, model access, Guardrail implementation, Memory and observability code must be ready.

02 / Activate build tools
Verify Python, uv, Node, npm and AgentCore CLI. For this run the CLI version was 0.30.0; pin the tested version when reproducing.

03 / Stage through the project helper
deploy_to_agentcore_runtime stages code under build/runtime and configures PUBLIC networking, HTTP and the foundation execution role.

04 / Deploy through CLI
Run the project deploy command. It uses the supplied CLI helper and CloudFormation rather than a direct create_agent_runtime SDK call.

05 / Save actual outputs
Write the returned runtime ARN, guardrail ID and numbered version to .env. Do not copy an old version from an earlier screenshot.

06 / Invoke the hosted application
Use invoke to exercise the deployed runtime, then inspect its response and trace ID.

```
python src/agent_orchestrator.py deploy
python src/agent_orchestrator.py invoke \
  "What is the return policy for premium customers?"
```

Required runtime environment: AWS_REGION, PROJECT_NAME, RETURNS_KB_ID, SHIPPING_KB_ID, WARRANTY_KB_ID, AGENT_LOG_GROUP, GUARDRAIL_ID and GUARDRAIL_VERSION. The supplied model wrapper attaches the configured guardrail.

Deployment evidence - what success means

```
Outputs:
  ApplicationAgentAgentcoreRuntimeRuntimeIdOutput11AAB7A8: udacity_agentcore_runtime-Oqj8s9Emc3
  ApplicationAgentAgentcoreRuntimeRuntimeArnOutput82AE9F7C: arn:aws:bedrock-agentcore:us-east-1:844274145783:runtime/udacity_agentcore_runtime-Oqj8s9Emc3
  ApplicationAgentAgentcoreRuntimeRoleArnOutput1A532F46: arn:aws:iam::844274145783:role/udacity-agentcore-agentcore-role
  StackNameOutput: AgentCore-udacity-default

Note: Transaction search enabled. It takes ~10 minutes for transaction search to be fully active and for traces from invocations to be indexed.
Next: agentcore invoke | agentcore status

Log: agentcore/.cli/logs/deploy/deploy-20261024-060626.log
  Waiting for runtime status READY ready.
CloudWatch Logs enabled at INFO: /aws/bedrock/agentcore/udacity-agentcore
X-Ray tracing enabled with 100% sampling

Step 6/6: Deploying AgentCore Gateway...
  [Skipped] Gateway: No configured Lambda tool functions are available.

============================================================
  Deployment Complete!
============================================================

  Add these to your .env file:
  AGENTCORE_RUNTIME_ARN=arn:aws:bedrock-agentcore:us-east-1:844274145783:runtime/udacity_agentcore_runtime-Oqj8s9Emc3
  GUARDRAIL_ID=7aqocuesx9r2
  GUARDRAIL_VERSION=5

  Then try the deployed runtime:
  python src/agent_orchestrator.py invoke "What is the return policy for premium customers?"
  or with the CLI:  agentcore invoke "What is the return policy for premium customers?"
  (agentcore status / agentcore logs show the deployed runtime and its logs)

(.venv) novamart-work $
```

Actual user screenshot: redeployment completed, runtime READY, INFO logs and 100% tracing enabled. Guardrail version 5 was reported.

Read the output in order

Stack output: AgentCore-udacity-default identifies the deployment. **READY:** control-plane provisioning completed. **Logging/tracing:** configuration was applied. **.env outputs:** values the local invoke/test commands must use.

READY is not the same as a working model request. In this project, a previously READY runtime still returned HTTP 500 because model access was blocked. Always follow deployment with an actual invocation.

Local versus deployed execution

Command	Where the agent application runs
python src/agent_orchestrator.py test	Local Python in CloudShell; it calls AWS services and publishes traces.
python src/agent_orchestrator.py invoke ...	Calls your already deployed AgentCore runtime.
python tests/test_agent.py all	Official project scoring checks across code and AWS resources.

Lab 9 - logging and X-Ray

01 / Configure logging
Build loggingConfiguration: cloudWatchConfig points to config.AGENT_LOG_GROUP, INFO, enabled=True.

02 / Configure tracing
Set xRayConfig enabled=True and samplingRate=1.0. Pass the dictionary to apply_observability_config inside try/except; do not hide failures.

03 / Generate traffic
Run test for the scripted scenarios, or invoke for a hosted request. Record the printed X-Ray ID.

04 / Open the map
CloudWatch > X-Ray traces > Service map or Trace Map, region us-east-1. Choose the time range containing your requests and refresh.

05 / Inspect the graph
Find NovaMart-Orchestrator, workers and three KnowledgeBase nodes. Inspect traces to distinguish a single request from a map aggregating several requests.

06 / Capture evidence
Use full-screen view; center the graph and make the labels readable. Preserve actual screenshots.

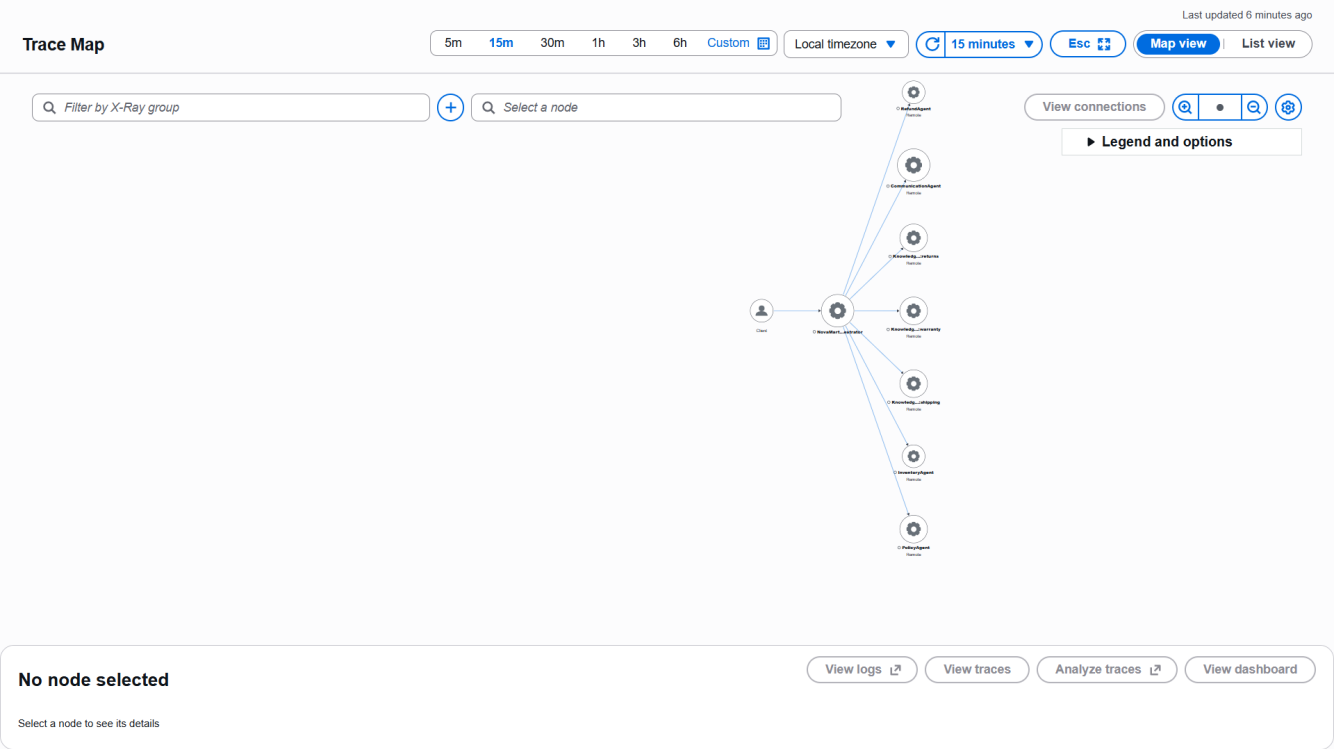
```
python tests/test_agent.py task6
python src/agent_orchestrator.py test 2>&1 | tee live-scenarios.txt
```

Two log groups to recognize

The configured application group is **/aws/bedrock/agentcore/udacity-agentcore**. Runtime stdout/stderr also appeared under **/aws/bedrock-agentcore/runtimes/<runtime-id>-DEFAULT**. If one group is empty, inspect the runtime group before changing application code.

```
aws logs describe-log-groups \
  --log-group-name-prefix /aws/bedrock-agentcore/ \
  --region us-east-1 --output table --no-cli-pager
```

Your actual X-Ray map



Actual AWS Console screenshot supplied by the user. It is an aggregate trace/service map, not the conceptual architecture illustration.

Visible node	How to interpret it
NovaMart-Orchestrator	Entry request and central routing activity.
Inventory / Refund	Order facts and return-decision work.
PolicyAgent	Policy coordination work.
CommunicationAgent	Final response composition.
KnowledgeBase nodes	Returns, shipping and warranty retrieval activity.

The screenshot shows KnowledgeBase nodes connected to the root alongside workers. The supplied instrumentation determines how those segments appear; this map alone does not prove a PolicyAgent-to-KB parent-child edge. Use individual trace timelines to inspect nesting and parallel timing.

Troubleshooting lab - find the real cause

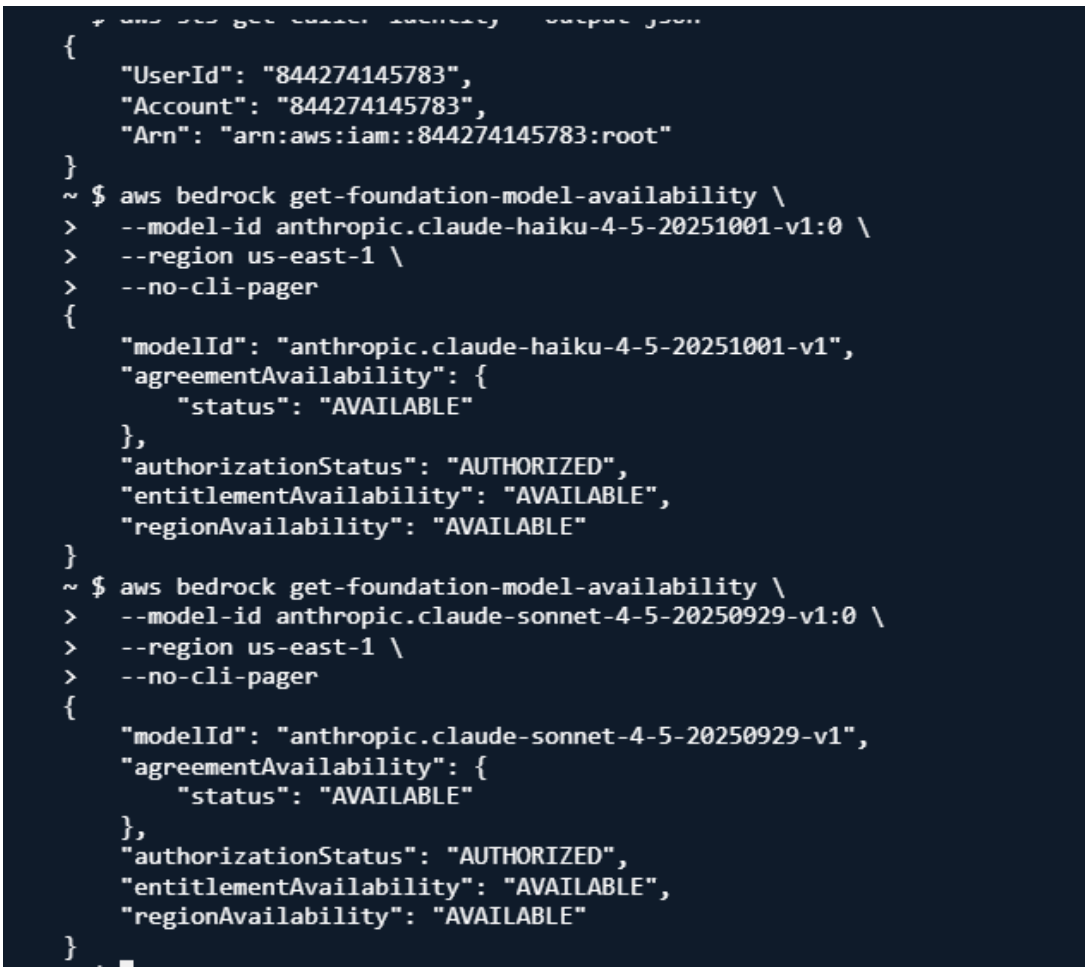
Symptom	Diagnosis and next action
HTTP 500 after READY	Read runtime CloudWatch traceback; do not assume deployment failure or redeploy blindly.
Marketplace AccessDenied	Check actual model availability and the calling identity. Complete required access setup using an authorized principal.
No logs in app group	List runtime log groups and tail the matching runtime group.
ENOSPC during npm build	Check df -h. Put CLI, npm cache and build workspace under /tmp; keep source/.env backed up in home.
tsc: command not found	In agentcore/cdk, install development dependencies using npm ci --include=dev.
uv is required	Check command -v uv. Activate the intended environment or install uv there.
Only Node version warning	Read the detailed failure. In this run uv absence caused synthesis failure, not that warning.
/tmp project missing	CloudShell session restart removed temporary workspace. Resume from the home project.
CLI says Local only	Local deployment metadata may be missing; inspect existing stack and preview diff before applying.
Expected prompt not found	Read the current function; the edit may already be present. Never apply an unchecked replacement.
ZIP Permission denied	ZIP path is not a command. Use CloudShell Actions > Download file.

Read-only first diagnostics

```
pwd
command -v python
command -v uv
df -h "$HOME" /tmp
aws sts get-caller-identity
agentcore status
agentcore deploy --target default --diff
```

A diff can show stale environment settings. In your run it proposed removing tracing variables and changing Guardrail 4 to 2. The project deployment function restaged code and reapplied observability; the successful update reported version 5.

Troubleshooting evidence - model access



Actual read-only availability checks: both project Claude models reported agreement/entitlement/region AVAILABLE and authorization AUTHORIZED.

A disciplined diagnostic sequence

01 / Read the final traceback line
The runtime error named Marketplace subscription actions. Treat that as a clue, not proof of the exact policy problem.

02 / Identify the caller
STS showed root for the Console/CloudShell session. The runtime itself uses its separate execution role.

03 / Check availability
Inspect the four status fields for both model IDs. If ERROR is returned, read errorMessage.

04 / Retry the application
After access was available, the deployed policy request succeeded without a source redeployment for that access issue.

Do not grant broad permissions merely because an error mentions IAM. Account entitlements, agreements, organization restrictions and the actual caller can change the diagnosis.

Lab 10 - correct monetary rounding

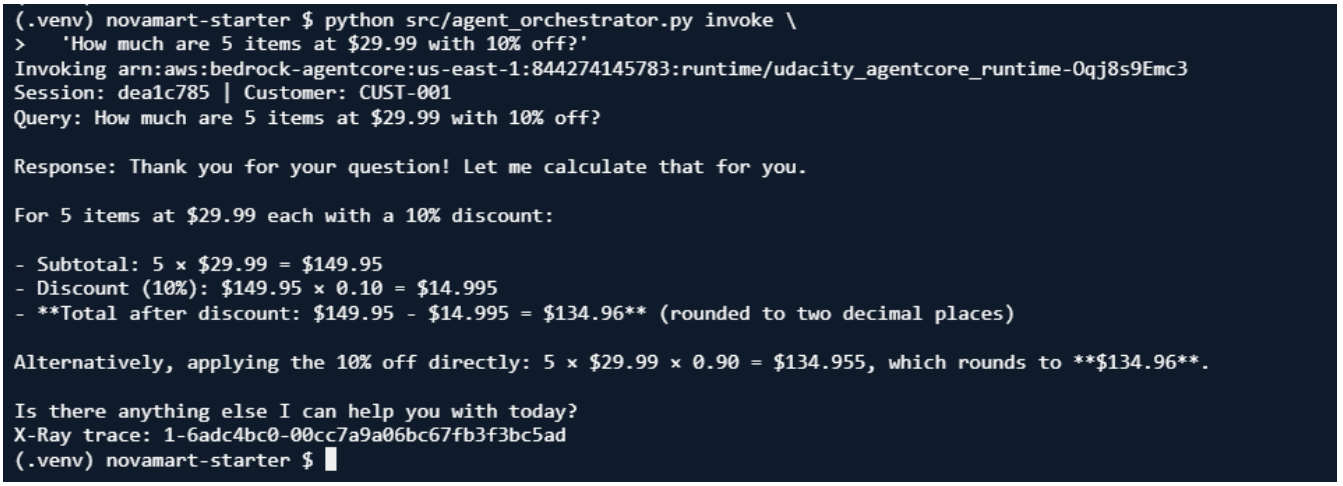
The failure

The model rounded the discount of $14.995 to $15.00, then subtracted it from $149.95 and answered $134.95. The intended rule was to keep precision through intermediate steps and round only the final payable total.

```
Subtotal: 5 * 29.99 = 149.95
Exact total: 149.95 * 0.90 = 134.955
Final total (ROUND_HALF_UP): 134.96
```

What you changed

CommunicationAgent was instructed to keep full decimal precision, apply the discount before rounding, and round the final amount to two decimals using ROUND_HALF_UP. You tested locally, redeployed, and verified the hosted response.



Actual deployed invocation after the prompt update: final total $134.96 and an X-Ray trace ID.

Stronger production implementation - extension exercise

```
from decimal import Decimal, ROUND_HALF_UP

total = (Decimal("29.99") * 5 * Decimal("0.90"))
rounded = total.quantize(Decimal("0.01"),
                         rounding=ROUND_HALF_UP)
assert rounded == Decimal("134.96")
```

This Decimal snippet is an extension, not a claim that it was wired into your submitted agent. A prompt example improves behavior but cannot guarantee arithmetic. For production, expose a validated deterministic calculation tool and have Communication use its result. Construct Decimal from strings, not binary floats.

Final lab - verify and submit

01 / Run official tests
Execute the complete suite after final source/deployment changes. Preserve the output and inspect any failures, not only the numeric score.

02 / Run the three scenarios
Return ORD-27176 as CUST-001; Premium return policy; 5 items at $29.99 with 10% off. Confirm route and evidence, not just fluent text.

03 / Capture test evidence
Screenshot the actual 120/120 result. Your final run showed the score and All tasks complete.

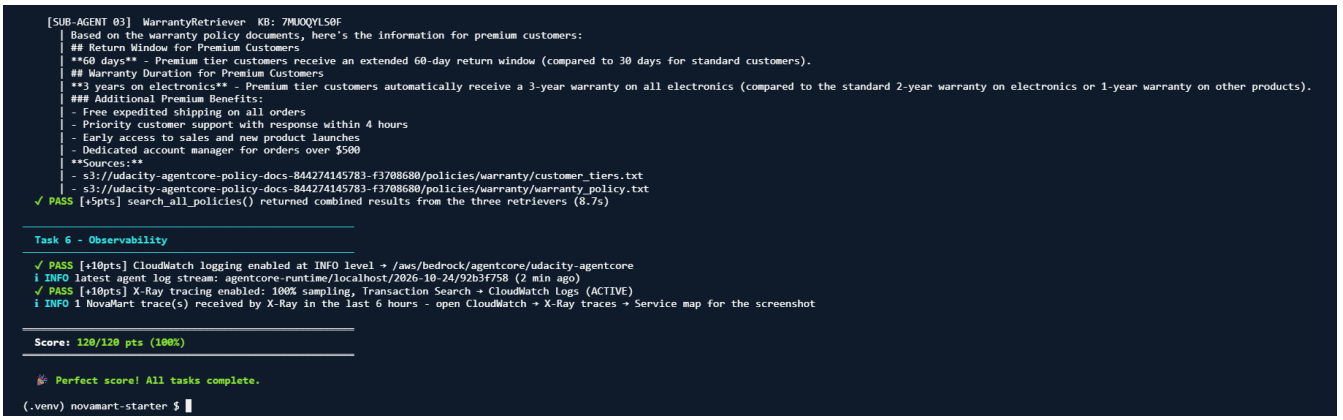
04 / Capture trace evidence
Include the real Console map with Orchestrator, workers and KnowledgeBase nodes.

05 / Package exact deliverables
Include src/agent_orchestrator.py and populated .env. Add logs as supporting evidence. Keep AWS credentials out of the package.

06 / Download and inspect
Use Actions > Download file. Open the ZIP and confirm expected paths before uploading to the course portal.

```
python tests/test_agent.py all 2>&1 | tee test-results-final.txt

zip "$HOME/NovaMart_Submission.zip" \
  src/agent_orchestrator.py .env test-results-final.txt
```



Actual final test screenshot supplied by the user: 120/120 points (100%), Perfect score! All tasks complete.

Your final recorded Guardrail version was 5. Always use the current deployment output rather than a historical version in this workbook. Keep resources until submission is complete; review cleanup scope before deleting anything.

Practice worksheet and reference shelf

Explain the design aloud

Prompt: Why must Inventory read both customer_id and order_id?
Notes: _____

Prompt: How does expected_version prevent one worker overwriting another?
Notes: _____

Prompt: Why does a tier question go to Inventory instead of Policy?
Notes: _____

Prompt: How does as_completed differ from waiting in submission order?
Notes: _____

Prompt: Why can READY coexist with HTTP 500?
Notes: _____

Prompt: What is the difference between WorkflowState and conversational Memory?
Notes: _____

Prompt: Why does 120/120 not guarantee a correct money calculation?
Notes: _____

Optional extension labs

1. Add a Decimal calculation tool and test half-cent boundaries. **2.** Wire Memory event writes/retrieval and verify same-session recall and cross-customer isolation. **3.** Inspect one X-Ray trace timeline to verify parallel KB calls. **4.** Add an authenticated customer boundary and an idempotent payment-provider sandbox refund.

Official resources and source notes

Starter project

Strands Bedrock model provider

Bedrock Knowledge Bases creation

Bedrock model access

AgentCore Runtime

AgentCore Memory

AgentCore observability

AgentCore CLI

Sources: assignment instructions, visible implementation discussion and user-supplied screenshots from this project. Official links checked on 24 October 2026; SDK defaults and Console labels can change. No new deployment or scoring run was performed while authoring this workbook.